DressMe

Architecture des agents IA

Dossier de validation technique

5 agents LangGraph, 3 pipelines LLM, 1 RAG, 4 services, 11 ecrans, protocole A2A

Graphes, prompts, outils, integration, garde-fous et limites

Projet de fin d'etudes - application DressMe (React, FastAPI, LangGraph, OpenAI, Supabase)

1. Vue d'ensemble

DressMe est une application React 19 qui parle a **quatre services FastAPI** (BO1, BO3, Help Me Pack, RAG). Supabase fournit l'authentification, la base de donnees (garde-robe) et le stockage des images. OpenAI fournit le LLM, Open-Meteo la meteo reelle.

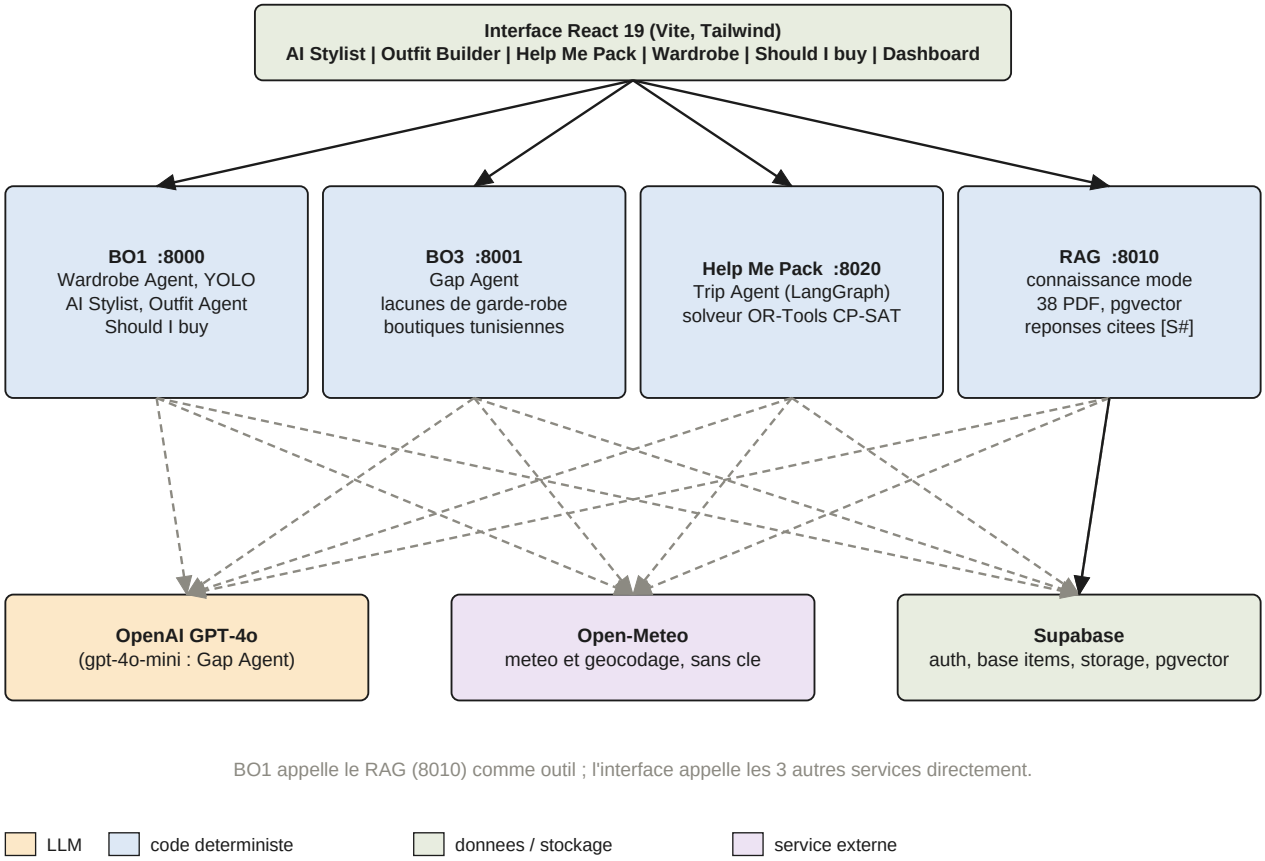


Figure 1 : architecture globale, services et dependances externes.

Principe commun a tous les agents : le LLM decide et redige, le code deterministe verifie et calcule. Aucun agent ne peut citer une piece absente de la garde-robe, inventer la meteo ou inventer une source.

Ports et lancement

Service	Port	Role	Lancement
BO1 (backend/app)	8000	garde-robe, detecteur, AI Stylist, Outfit Agent, Should I buy, avatar	start_bo1.ps1
BO3 (DressMe UI Blueprint/backend)	8001	Gap Agent, produits de boutiques tunisiennes	start_bo3.ps1
Help Me Pack (backend/main.py)	8020	Trip Agent et solveur CP-SAT	start_packing.ps1
RAG (rag_backend)	8010	connaissance de mode citee	start_rag.ps1
Interface (Vite)	8443	application React (https de developpement)	pnpm dev

2. Integration de toutes les fonctionnalites

Chaque page de l'interface appelle une route ; la route appelle soit un **agent** (le LLM choisit des outils ou suit un graphe), soit un **moteur** (detecteur, moteur d'achat, comparaison locale). Le schema relie les 11 ecrans aux 5 agents (tous LangGraph) et aux moteurs.

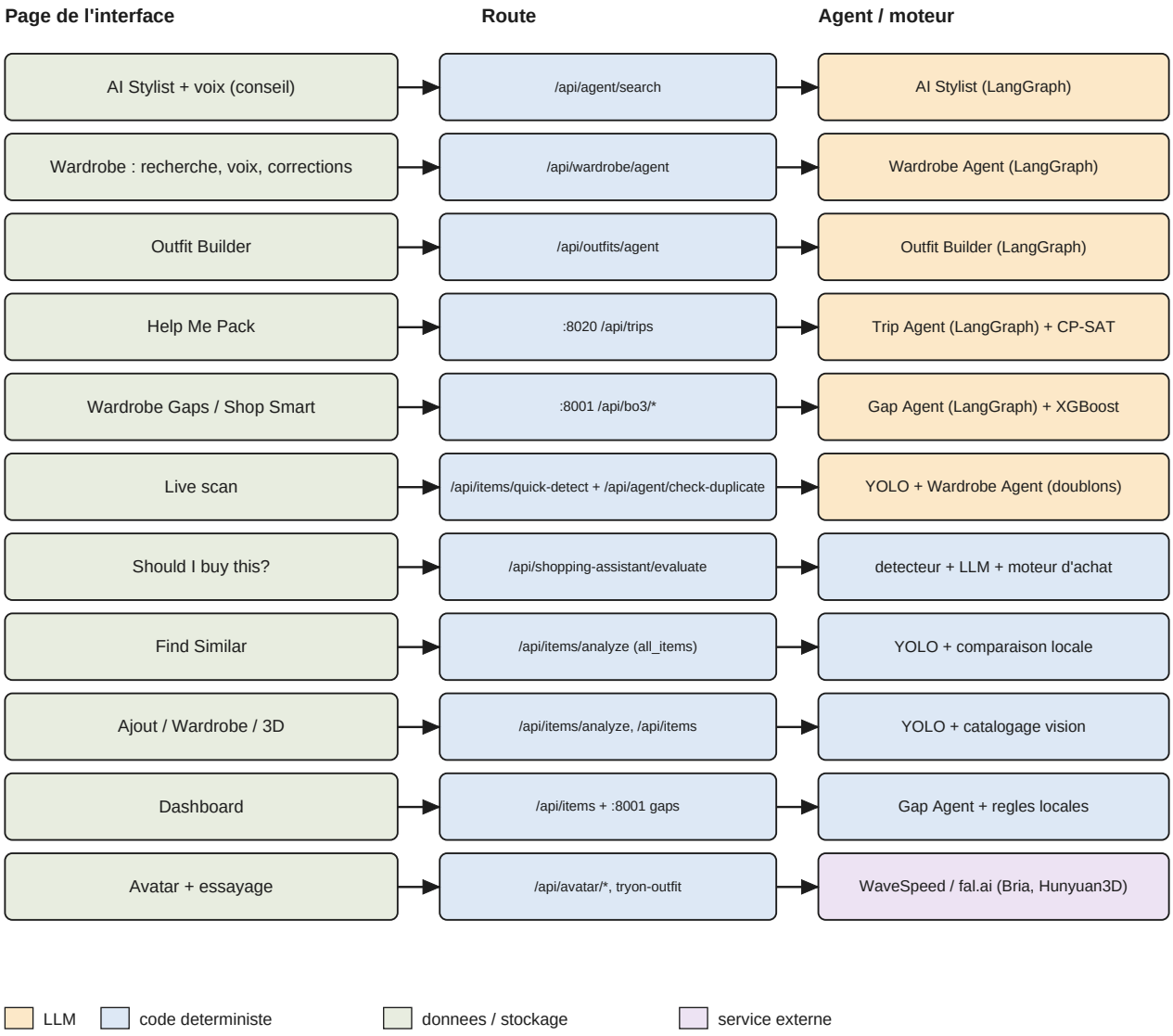


Figure 2 : de chaque page a l'agent ou au moteur qui la sert.

2.1 Recherche vocale



Transcription faite par le navigateur (Chrome et Edge utilisent leur service en ligne) : seul le texte arrive a nos serveurs.

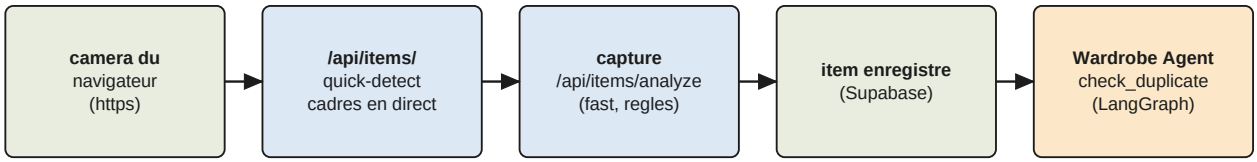
Voix ou texte : meme chemin. Le LLM choisit les filtres, le code filtre la garde-robe, la page affiche le resultat.

Figure 3 : recherche par la voix ou par le texte (Wardrobe Agent).

- **C'est bien un agent.** Le texte (ecrit ou dicte) est envoye au LLM, qui choisit lui-meme les parametres de recherche (categorie, couleur, saison, occasion, style, chaleur, texte libre), appelle l'outil `search_wardrobe`, **regarde le resultat** et recommence avec d'autres filtres si c'est vide ou trop large, puis redige sa reponse.

- Voix : hook `useSpeech` (Web Speech API du navigateur, transcription en direct) ; a la fin de la phrase le texte part comme une question ecrite.
- Page Wardrobe : la grille est filtree sur les pieces trouvees et les **filtres choisis par le LLM** s'affichent en pastilles ("jacket, warmth 4+"). Navigateur sans l'API vocale : le bouton micro est masque.
- **Le Wardrobe Agent** (LangGraph) sert cette recherche, mais aussi le comptage ("combien de hauts ?"), la recherche de pieces proches et les corrections, qui ne sont jamais enregistrees sans la confirmation de l'utilisateur (voir section 8.1).

2.2 Live scan



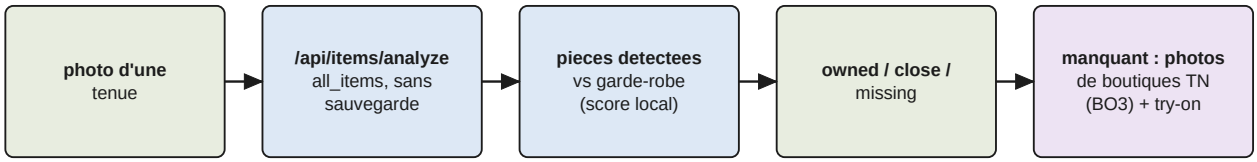
Pendant la camera : YOLO dessine les cadres et signale une photo floue ou un vetement coupe (module quality).

Apres chaque capture, l'agent decide avec search_wardrobe s'il s'agit du meme vetement scanne deux fois.

Figure 4 : scan en direct avec controle qualite et verification de doublon.

- La camera exige une origine securisee (https ou localhost) : le serveur de dev peut tourner en https avec un certificat local.
- `/api/items/quick-detect` renvoie les cadres, la couleur principale et un eventuel probleme (flou, coupe, trop petit).
- La capture passe par `/api/items/analyze?fast=true` (classification par regles, quelques centaines de ms), puis le Wardrobe Agent (outil check_duplicate) decide s'il faut avertir.

2.3 Find Similar



Aucun LLM : detecteur YOLO + comparaison par categorie, couleur, matiere et style cote interface.

Une piece manquante renvoie vers BO3 (/api/bo3/plan/photos) et vers l'essayage virtuel sur l'avatar.

Figure 5 : de la photo d'une tenue aux pieces manquantes.

- Une photo de tenue est decoupee en pieces par le detecteur ; chaque piece est comparee a la garde-robe (meme categorie, couleur, matiere, style) et classee **You own it**, **Close match** ou **Missing**.
- Pour une piece manquante, BO3 renvoie des photos de boutiques tunisiennes et l'utilisateur peut l'essayer sur son avatar avant d'acheter.
- Difference avec Should I buy : Find Similar part d'une tenue vue ailleurs ; Should I buy part d'un seul article et rend un verdict.

2.4 Dashboard

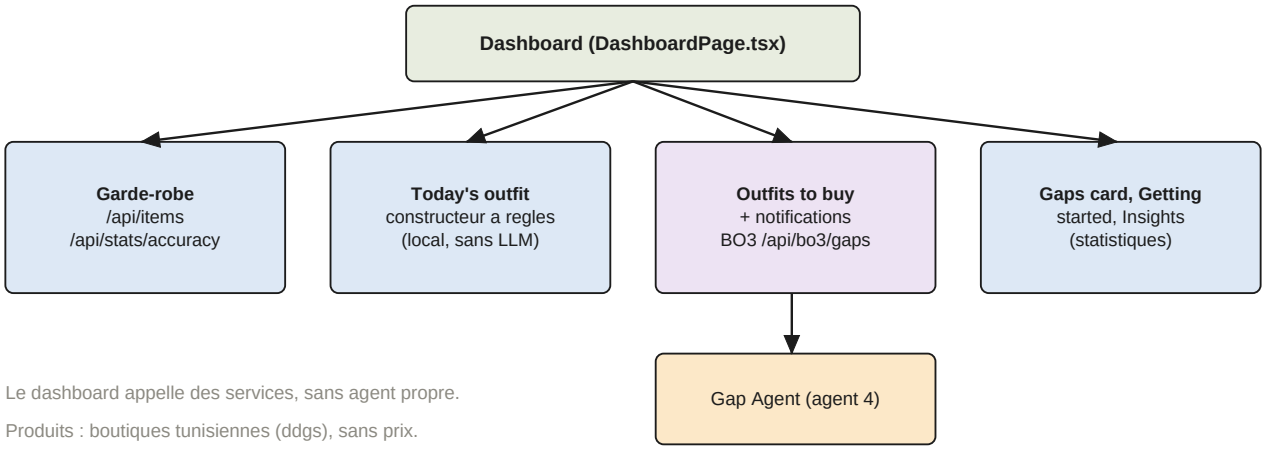


Figure 6 : composition du tableau de bord.

- **Outfits to buy** et les notifications viennent du Gap Agent (BO3) ; le panneau memorise l'objectif choisi pour prevenir quand des lacunes changent.
- **Today's outfit** utilise le constructeur a regles local (une tenue pour la saison courante) ; le Outfit Builder Agent est sur sa propre page.
- Les statistiques (nombre de pieces, repartition, precision du catalogage) viennent de `/api/items` et `/api/stats/accuracy`.

2.5 Essayage virtuel et avatar

- Avatar : photo en pied, puis modele 3D (Hunyuan3D v3) ; fournisseur configurable (WaveSpeed par defaut ou fal.ai, variable AVATAR_PROVIDER).
- Essayage d'une tenue, d'un vetement de boutique ou d'une tenue de l'Outfit Builder : `/api/avatar/tryon-outfit` et `tryon-garment` (Bria virtual try-on), execute en tache d'arriere-plan avec suivi d'etat.

2.6 Services partages par les agents

Service	Utilise par	Role
Supabase (auth, items, storage)	tous	jeton verifie sur chaque route, garde-robe de l'utilisateur, photos et cutouts
OpenAI GPT-4o	AI Stylist, Wardrobe Agent, Outfit Agent, Trip Agent, catalogage, Should I buy	raisonnement, redaction, vision
Open-Meteo	AI Stylist, Outfit Agent, Trip Agent	geocodage + prevision 16 jours, cache 30 min
RAG (8010)	AI Stylist, Outfit Agent	conseils de livres de mode avec sources [S#]
YOLO dressme_best.pt	Wardrobe, Live scan, Find Similar, Should I buy	detection et segmentation des vetements
OR-Tools CP-SAT	Trip Agent	choix optimal de la valise
XGBoost	Gap Agent, Should I buy	classement des lacunes, decision d'achat
WaveSpeed / fal.ai	Avatar, essayage	generation 3D et try-on

3. Combien d'agents ?

5 agents, tous construits avec **LangGraph**, sont appeles depuis l'interface. **3 pipelines** utilisent un LLM sans etre des agents (le LLM ne choisit pas ses outils). Une **pile separee de l'equipe** (Stylist BO4 et orchestrateur) existe aussi, non branchee a l'interface React.

#	Agent	Fonctionnalite	Technique	Fichier
1	AI Stylist	page AI Stylist (texte et voix)	graphe : boucle llm / tools, rappel unique, 3 outils	backend/app/agent.py
2	Wardrobe Agent	recherche texte et voix, comptage, pieces proches, corrections confirmees, doublons du Live scan	graphe : gate, apply / cancel / assist ; boucle d'outils, 4 outils	backend/app/wardrobe_agent.py
3	Outfit Builder Agent	page Outfit Builder	graphe 5 noeuds : weather, closet, knowledge, compose, validate	backend/app/outfit_agent.py
4	Trip Agent	Help Me Pack	GPT-4o, 10 outils, 2 graphes, CP-SAT	packing_backend/trip_agent/
5	Gap Agent (BO3)	Wardrobe Gaps, Shop Smart, Outfits to buy	boucle llm / tools (agent_kit), 5 outils, XGBoost	DressMe UI Blueprint/backend/agent/gap_agent.py

Pipeline LLM	Role	Fichier
Catalogage vision	detecteur YOLO, puis GPT-4o vision (JSON strict) : categorie, couleur, matiere, saison, occasion	backend/app/enricher.py
Should I buy this?	detecteur + LLM, moteur d'achat (XGBoost + regles), puis LLM qui redige 3 sections	smart_purchasing/inference.py
Reponse RAG	recherche hybride, puis LLM qui repond uniquement avec les sources	rag_backend/rag/rag_service.py

Pile separee (non branchee a React)	Technique
Stylist BO4 (bo4/)	LangGraph a 9 noeuds, Ollama qwen2.5:3b, moteur de tenues heuristique (couleur 30, style 20, occasion 30, saison 20)
Orchestrateur (orchestrator/, port 8070)	LangGraph : route_intent (regles puis LLM local) vers stylist, garde-robe, achat, RAG ou clarification

Tous les agents de l'application sont des graphes LangGraph. La boucle d'appel d'outils est commune : le module `agent_kit/tool_loop.py` (noeuds llm, tools, nudge, finish ; chaque agent branche son LLM, ses outils et ses garde-fous). Le Trip Agent et l'Outfit Builder ont en plus des graphes metier (workflow deterministe, composition puis validation).

4. Outils de chaque agent

Un **outil** est une fonction que le LLM peut demander ; le code l'execute et renvoie le resultat au LLM. Le LLM ne touche jamais directement aux donnees.

4.1 AI Stylist : 3 outils

Outil	Parametres	Retour	Garde-fou
search_wardrobe	category, category_group, color_name, season, occasion, style, pattern, min_warmth (1-5), text (choisis par le LLM, voix ou texte)	jusqu'a 20 pieces : id, nom, categorie, couleur, matiere, saisons, style, occasions, formalite, chaleur	valeurs d'enumeration imposees ; seuls les items de l'utilisateur
get_weather	days_ahead (0 a 15), city (optionnel)	ville, jour, condition, min/max en C et F, % de pluie, mm, vent, resume	ville inconnue : premiers mots, puis Tunis, annoncee ; sinon "non verifiee"
fashion_advice	question (anglais, un seul sujet)	answer + sources [{marker, title, page}]	None si RAG arrete ou sans citation : l'agent le dit

4.2 Outfit Builder Agent : 5 noeuds (pas d'appel d'outils)

Noeud	Entree	Sortie	Type
weather	ville, pays	forecast (ou None)	code, Open-Meteo
closet	items	closet : pieces par role avec 13 champs	code
knowledge	occasion, saison, forecast	advice[] + sources	RAG, 2 questions
compose	brief : occasion, meteo, conseils, tenues deja vues, placard	outfits[] {title, items[ids], weather_note, why[]}, missing[]	GPT-4o JSON, temp. 0,6
validate	outfits[] + placard	tenues valides ou liste d'erreurs (retour a compose une fois)	code

4.3 Trip Agent : 10 outils (valides par Pydantic)

Outil	Role	Ressource
get_trip_details	extrait, fusionne et valide les champs du voyage ; signale ceux qui manquent	GPT-4o JSON + regex de secours
get_weather	prevision jour par jour pour la destination et les dates (force_refresh, allow_network)	Open-Meteo, repli synthetique etiquete
get_wardrobe	charge la garde-robe (filtres category, style, limit) et la couverture des roles	CSV + compatibilite (pandas)
get_activity_requirements	traduit activites, meteo et formalite en exigences quotidiennes du solveur	regles deterministes
optimize_trip	choisit la valise : moins de pieces, plus combinables, poids minimal	OR-Tools CP-SAT
generate_checklist	cree la checklist (tout non emballe) depuis la capsule	modeles Pydantic
suggest_alternatives	diagnostique INFEASIBLE et propose des options sans relacher les contraintes	CP-SAT + analyse des roles
update_trip	modifie dates, destination, activites, tenue ou poids ; invalide l'etat obsolete ; re-optimise au choix	etat de l'agent
update_checklist	coche ou decoche une piece sans relancer le solveur	etat de l'agent
check_gaps	crochet optionnel vers le Gap Agent (peut repondre indisponible)	fonction injectee

4.4 Gap Agent BO3 : 5 outils

Outil	Parametres	Role
get_wardrobe	aucun	profil de style + resume de chaque piece possedee
compute_coverage	goal (enum des objectifs)	exigences couvertes ou manquantes ; les regles decident si une lacune existe
rank_gaps	goal	score chaque exigence manquante (deficit, frequence d'usage, saison, gain de tenues, preference, redondance FashionCLIP) puis classe avec XGBoost ; sauvegarde l'analyse
get_past_analyses	aucun	memoire a long terme : analyses precedentes de l'utilisateur
show_items	item_ids (8 max)	affiche les pieces citees comme cartes photo a cote de la reponse

Le Gap Agent a une vraie **memoire** (analyses passees) et un **apprentissage** : les retours de l'utilisateur sur les suggestions reentrainent le classeur (/api/bo3/retrain).

4.5 Wardrobe Agent : 4 outils, aucun n'ecrit

Outil	Parametres	Role
search_wardrobe	filtres choisis par le LLM (categorie, couleur, saison, occasion, style, chaleur, texte)	pieces de l'utilisateur ; sert aussi aux autres agents
find_similar	item_id, ou category, color_name, material, style	pieces les plus proches, avec les raisons (meme type, meme couleur...)
wardrobe_stats	aucun	total et repartition par groupe, categorie, famille de couleur, saison, occasion ; pieces a verifier
update_item	item_id, fields	PROPOSE une correction : rien n'est enregistre

Une correction n'est **appliquee que par le noeud** `apply` du graphe, apres un oui explicite de l'utilisateur (message "oui" ou confirm=true dans la meme conversation). Le LLM n'a aucun outil d'ecriture. Un non, ou une nouvelle demande, annule la proposition ; elle expire apres 15 minutes et est liee a l'utilisateur et a la conversation.

Hors outils LLM, le meme agent expose des actions structurees sans LLM (data.action : list, search, check-duplicate), utilisees par les autres agents et par le Live scan.

5. AI Stylist

Entree : `POST /api/agent/search` avec message, city, country, unit. La ville du profil est utilisee, sinon **Tunis, Tunisie**. **Modele :** GPT-4o, temperature 0,3, sortie JSON, 8 tours maximum.

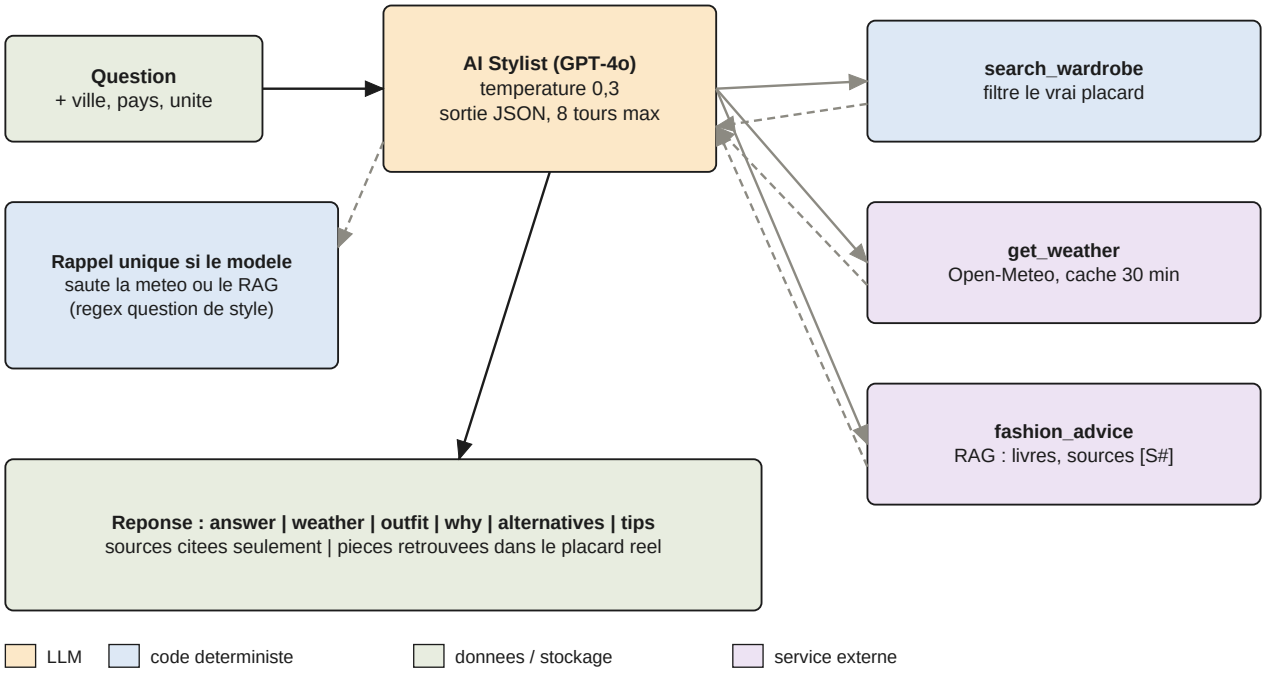


Figure 2 : boucle d'outils de l'AI Stylist.

Outils

Outil	Ce qu'il fait	Source
search_wardrobe	filtre la vraie garde-robe (categorie, couleur, saison, occasion, style, chaleur, texte)	table items (Supabase)
get_weather	prevision du jour voulu (aujourd'hui, demain, "vendredi"), en C ou F, 30 min de cache	Open-Meteo
fashion_advice	une question de style par sujet (jusqu'a 3), reponse avec [S#]	RAG (8010)

Prompt systeme (regles)

- Toujours appeler **search_wardrobe** avant de repondre ; ne parler que de pieces possedees ; si rien ne convient, le dire.
- **Meteo** : pour toute question "que porter / quelle tenue", appeler get_weather pour le jour voulu et citer les chiffres reels.
- **Livres** : pour toute question de style, appeler fashion_advice une fois par sujet (code vestimentaire, couleurs, tissus du climat).
- Ne jamais presenter un conseil de livre comme venant du placard ; ne jamais inventer de source.
- Position claire et convaincante : "My pick: ..." seulement pour une demande de conseil.

Format de reponse final (JSON, sans markdown ni emoji)

Cle	Contenu
answer	2 phrases : le choix puis la meilleure raison
weather	1 a 2 phrases : prevision chiffree et ce qu'elle impose
outfit	0 a 3 tenues, noms exacts des pieces reliees par " + "
why	4 a 6 raisons : meteo, occasion, couleurs, matieres, role des pieces ; chacune appuyee sur un fait reel et un [S#]
alternatives	0 a 3 lignes : ce qui a ete envisage et pourquoi ecarte
tips	2 a 3 conseils pratiques, avec [S#] quand ils viennent des livres

Garde-fous

- Rappel unique si le modele saute la meteo ou le RAG sur une question qui les demande (detection par expression reguliere FR/EN).
- Seules les sources reellement citees dans le texte sont renvoyees ; les pieces de la tenue sont retrouvees par leur nom dans le placard.
- Ville inconnue du service meteo : essai sur les premiers mots du nom, sinon meteo de Tunis annoncee comme telle.
- RAG ou meteo en panne : l'agent repond sans eux et le dit.

6. Outfit Builder Agent (LangGraph)

Route `POST /api/outfits/agent` (occasion, saison, ville, tenues deja montrees). Un graphe d'etat partage ou chaque noeud a un seul travail ; le LLM n'intervient que ou le jugement est necessaire.

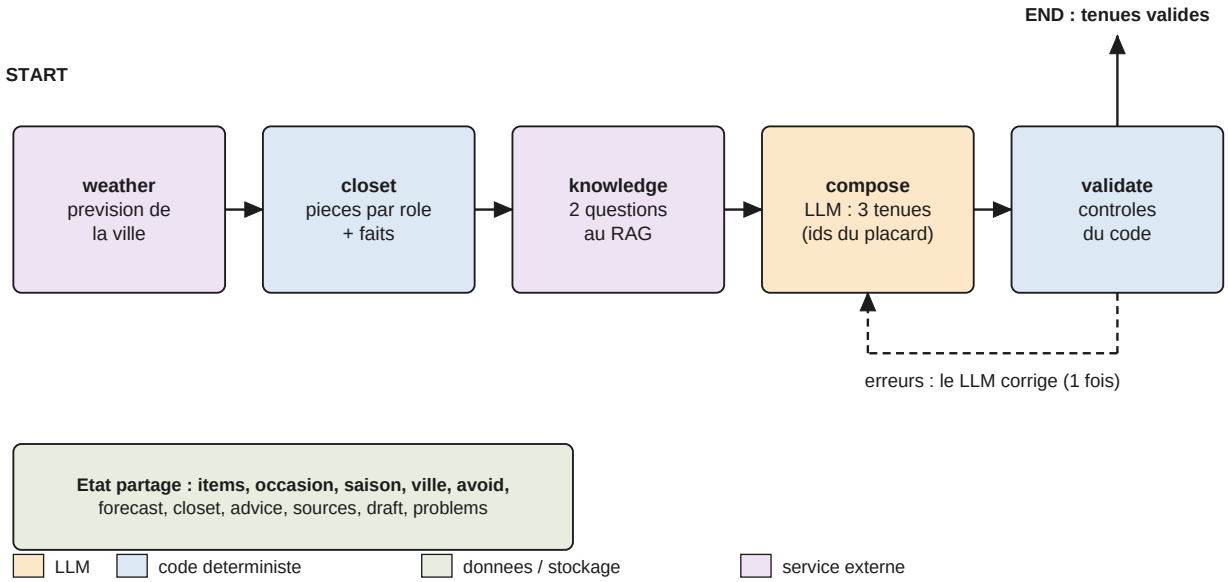


Figure 3 : graphe LangGraph de l'Outfit Builder.

Noeud	Role	LLM
weather	prevision reelle de la ville (Tunis par defaut)	non
closet	pieces classees par role : hauts, bas, robes, vestes, chaussures, avec couleur, matiere, saison, chaleur, formalite	non
knowledge	2 questions au RAG : code vestimentaire de l'occasion ; couleurs et tissus pour le climat du jour	RAG
compose	construit 3 tenues avec les identifiants du placard, donne un titre, une note meteo chiffree et 3 a 5 raisons	GPT-4o, JSON
validate	ids reels ; robe seule ou haut + bas ; chaussures ; pas deux pieces du meme role ; veste s'il fait froid ou pluvieux	non

Boucle de correction

Si une tenue est invalide, le LLM est renvoye **une seule fois** avec la liste exacte des erreurs ("Outfit 2: it has no shoes"). Seules les tenues valides sont retournees. Le bouton Shuffle envoie les tenues deja vues (`avoid`) pour en obtenir d'autres.

Interface et repli

- Bandeau meteo, titre de chaque look, phrase "Weather", liste "Why this outfit", bouton essayage virtuel, livres cites.
- Si l'agent est indisponible, l'ancien constructeur a regles (`outfitBuilder.ts`) prend le relais : la page affiche toujours quelque chose.
- Les pieces manquantes ("a rain jacket") ne sont signalees que si les conditions le justifient.

7. Trip Agent : Help Me Pack

Modele : GPT-4o, temperature 0, appel d'outils natif, 12 iterations maximum. Le LLM choisit l'outil suivant ; le code valide les arguments (Pydantic), impose les dependances et calcule la valise avec **OR-Tools CP-SAT**.

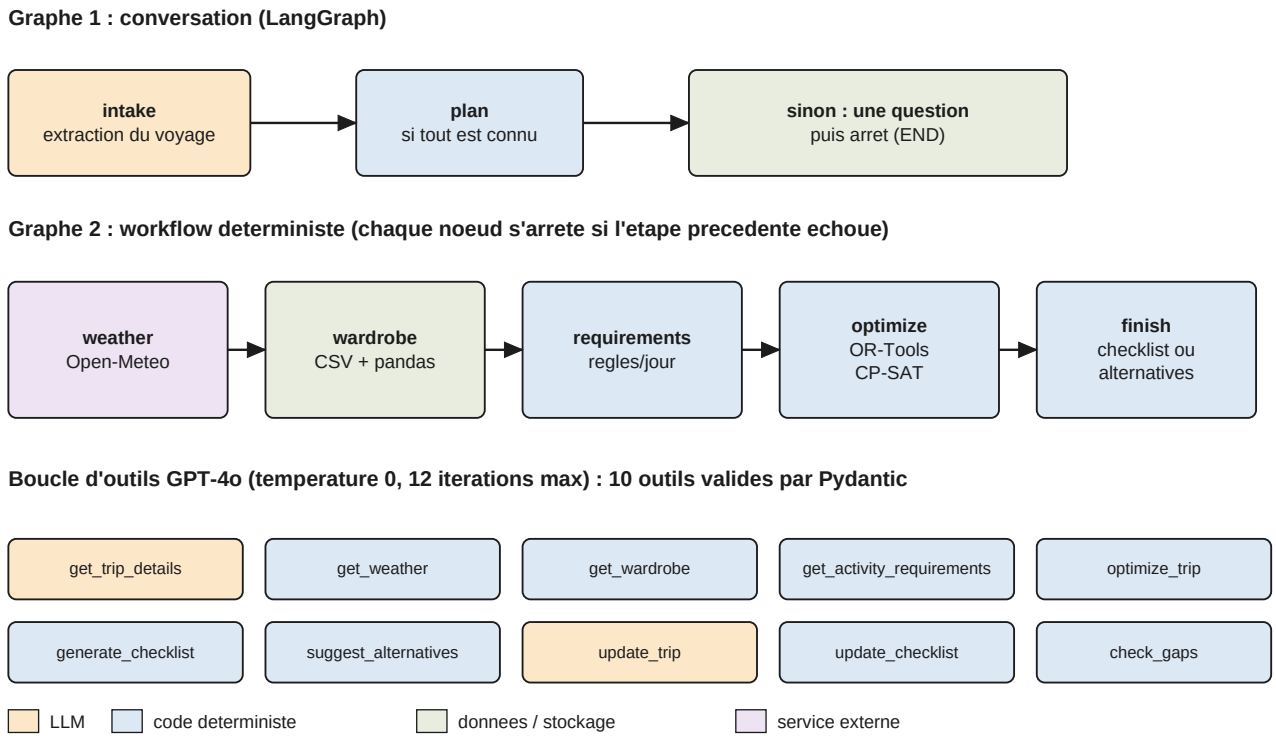


Figure 4 : deux graphes LangGraph et les 10 outils du Trip Agent.

Memoire

Type	Contenu	Duree
Travail	AgentState : voyage valide, meteo, placard, exigences, resultat du solveur, capsule, checklist, erreurs	une session
Conversation	16 derniers messages injectes dans le prompt	une session
Session	instance de l'agent par session_id, protegee par un verrou	jusqu'au redemarrage
Client	session_id dans le localStorage pour reprendre apres un rafraichissement	navigateur

Regles du prompt (10)

- Ne jamais inventer destination, dates, duree, activites, limite de bagage, meteo, pieces ni resultat du solveur.
- Voyage nouveau : get_trip_details d'abord ; s'il manque un champ, poser UNE question concise et s'arreter.
- Ordre : get_weather, get_wardrobe (une fois), get_activity_requirements, optimize_trip, puis generate_checklist.
- INFEASIBLE : suggest_alternatives ; ne jamais relacher silencieusement une contrainte dure.
- update_trip seulement pour modifier un voyage ; update_checklist pour cocher sans relancer le solveur.
- Une erreur d'outil se corrige ou s'explique, jamais un succes fabrique.

8. Gap Agent BO3 et boutiques tunisiennes

Modele : gpt-4o-mini par defaut, temperature 0,2, 6 etapes maximum. Il repond a "que manque-t-il dans ma garde-robe pour tel objectif ?" uniquement a partir des resultats d'outils.

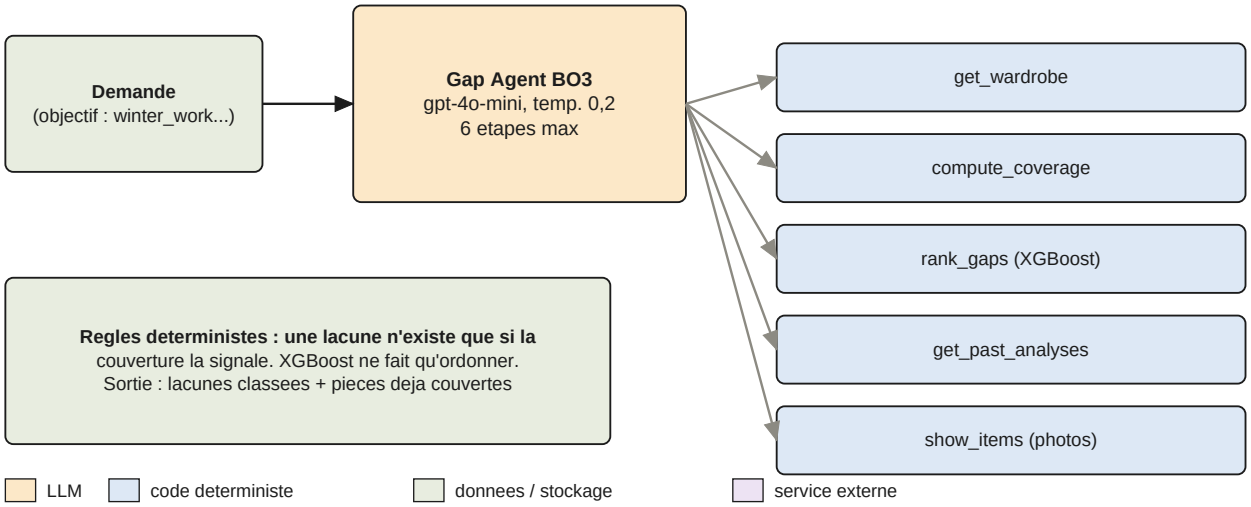


Figure 5 : Gap Agent et ses 5 outils.

- **Couverture** : regles deterministes par objectif et saison. Une lacune n'est reelle que si ces regles la signalent.
- **XGBoost** ordonne les lacunes ; son score n'est pas une probabilite, et il est entraine sur des donnees synthetiques tant qu'il n'y a pas de retours reels (le prompt le fait dire).
- **Departement** (femme, homme) : jamais de piece d'un autre departement ; jamais de genre devine.
- **Aval** : les lacunes alimentent "Outfits to buy" du tableau de bord, avec des produits de boutiques tunisiennes (recherche d'images ddgs) et l'essayage virtuel.
- Reponse dans la langue de l'utilisateur (anglais, francais, arabe tunisien).

8.1 Wardrobe Agent et detection des doublons (Live scan)

Le Wardrobe Agent possede la garde-robe. Il reprend la recherche texte ou voix et la detection de doublons du Live scan (GPT-4o, temperature 0,1 : il appelle `search_wardrobe` pour decider s'il s'agit du **meme** vetement scanne deux fois). Le graphe commence par un noeud **gate** : si une correction attend la reponse de l'utilisateur, un oui mene a **apply**, un non a **cancel** ; sinon **assist** lance la boucle d'outils. Les agents AI Stylist et Outfit Builder lisent le placard par lui (`list_items`), le Trip Agent le lui demande par A2A (action list, repli sur BO1).

9. Pipelines assistes par LLM

9.1 Catalogage d'un vetement



Sans cle API ou en cas d'erreur : regles de secours (fallback_enrich)

Figure 6 : de la photo a la fiche vetement.

GPT-4o ne change la categorie du detecteur que si l'image le montre clairement ; la couleur est choisie dans la palette la plus proche de la mesure sur les pixels ; les scores de confiance doivent etre honnetes (bas si l'image est ambigue).

9.2 Should I buy this?

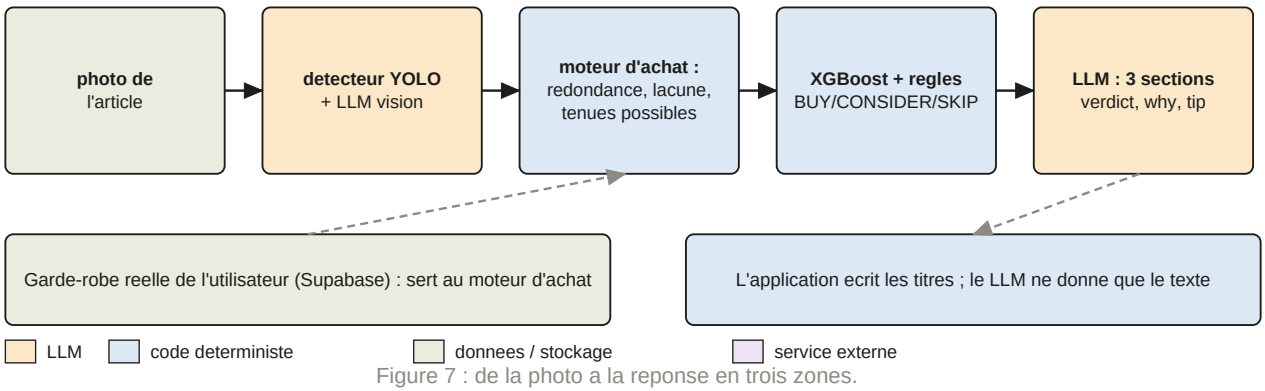


Figure 7 : de la photo a la reponse en trois zones.

- L'article est reconnu par le **meme** detecteur + LLM que le placard : pas de second moteur de vision.
- Le moteur d'achat calcule redondance, lacune comblee, tenues possibles avec les pieces possedees et prix par tenue.
- Le LLM (temperature 0,5) ne redige que trois sections : **verdict** ("Yes, buy it:", "Think twice:", "Skip it:" + raison), **why_it_works** (3 phrases avec un fait : nombre de tenues, nom d'une piece possedee, prix) et **final_advice**.
- Le prompt interdit le jargon (XGBoost, score, redondance), les emojis, le markdown et les symboles monetaires ; l'application ecrit les titres en gras.

10. Le RAG : connaissance de mode citee

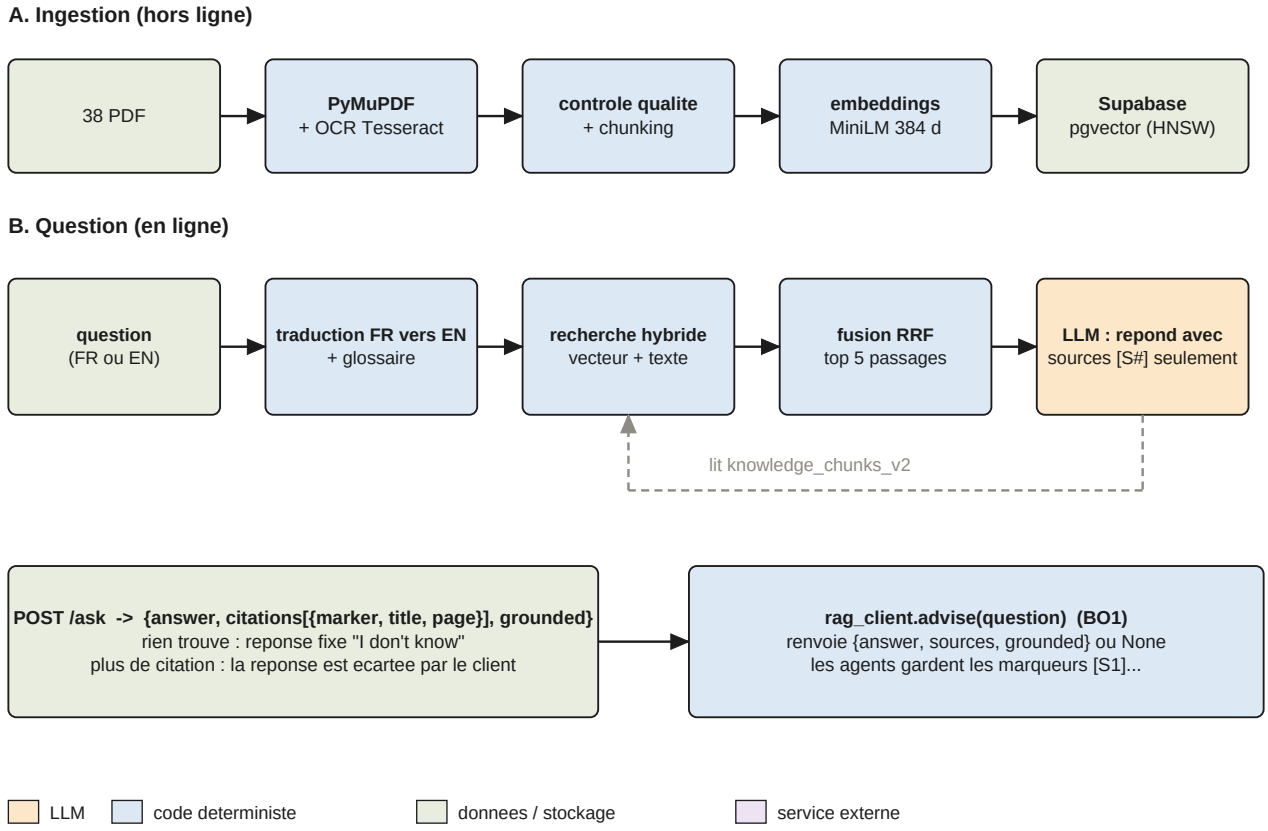


Figure 8 : ingestion et interrogation du RAG.

Element	Detail
Corpus	38 PDF (couleurs, tissus, codes vestimentaires, capsules, accessoires...), OCR Tesseract pour les livres scannes
Index	table knowledge_chunks_v2 (Supabase Postgres + pgvector, HNSW, cosinus), 384 dimensions, all-MiniLM-L6-v2
Recherche	hybride : vecteurs + recherche plein texte Postgres, fusion RRF, top 5 passages
Question	francais traduit en anglais (glossaire), anglais inchange
Generation	repond UNIQUEMENT avec les sources numerotees ; chaque puce finit par [S#] ; sinon la reponse fixe "I don't know"
API	POST /ask, /ask/stream (SSE), cle d'API, limitation de debit, historique de conversation

Integration dans les agents

- `rag_client.advise(question)` appelle /ask et renvoie {answer, sources:[{marker, title, page}], grounded}, ou None si le service est arrete, vide ou sans citation.
- L'AI Stylist l'expose comme outil **fashion_advice** ; l'Outfit Agent l'appelle dans le noeud **knowledge**.
- Les agents gardent les marqueurs [S#] dans leurs phrases ; l'interface les affiche en badges (survol : livre et page) et liste les livres sous "From fashion books".
- Le RAG ne connait pas le placard : c'est l'agent qui fait le lien entre la regle du livre et les pieces reelles.

11. Qui utilise quoi

Fonctionnalite	Agent / pipeline	Meteo	RAG	Placard	Solveur / ML
AI Stylist	AI Stylist	oui	oui	oui	non
Outfit Builder	Outfit Builder Agent	oui	oui	oui	non
Help Me Pack	Trip Agent	oui	hook prevu	oui	OR-Tools CP-SAT
Wardrobe Gaps, Shop Smart, Outfits to buy	Gap Agent	non	non	oui	regles + XGBoost
Should I buy this?	pipeline detecteur + LLM + moteur d'achat	non	non	oui	XGBoost
Live scan	detecteur + Wardrobe Agent (doublons)	non	non	oui	YOLO
Recherche, comptage, corrections	Wardrobe Agent	non	non	oui	non
Ajout d'un vetement	detecteur + catalogage vision	non	non	non	YOLO

Prompts : parametres LLM par composant

Composant	Modele	Temperature	Sortie	Limite
AI Stylist	GPT-4o	0,3	JSON	8 tours, 1800 tokens
Outfit Agent (compose)	GPT-4o	0,6	JSON	2 passes max, 2200 tokens
Trip Agent	GPT-4o	0	outils + JSON	12 iterations
Gap Agent	gpt-4o-mini	0,2	Markdown	6 etapes
Anti-doublon	GPT-4o	0,1	JSON	250 tokens
Should I buy (rédaction)	GPT-4o	0,5	JSON 3 sections	-
Catalogage vision	GPT-4o	-	schema JSON strict	-

Cycle d'une requete (exemple : "Que mettre demain pour un diner ?")

- 1. React envoie message + ville + unite du profil a BO1.
- 2. L'AI Stylist appelle get_weather (demain), search_wardrobe (saison, occasion) puis fashion_advice (code vestimentaire, couleurs).
- 3. GPT-4o redige le JSON : choix, meteo chiffree, tenue, 4 a 6 raisons, alternatives, conseils.
- 4. BO1 filtre les sources citees, retrouve les pieces dans le placard et renvoie la reponse + la prevision.
- 5. React affiche les zones Weather, Outfit idea (photos), Why I chose this, What else I considered, Tips, From fashion books.

12. Garde-fous, securite et tests

Theme	Mesure
Ancrage	identifiants du placard seulement ; outils en lecture ; sources numerotees ; meteo reelle ; "never invent" dans chaque prompt
Validation par du code	Outfit Agent (regles de composition), Trip Agent (Pydantic + solveur), AI Stylist (filtrage des sources citees)
Degradation douce	RAG, meteo ou LLM indisponibles : reponse partielle ou repli par regles, jamais une page cassee
Securite	jeton Supabase verifie sur chaque route ; chaque utilisateur ne voit que son placard ; cles API cote serveur uniquement ; limitation de debit du RAG
Honnetete	le prompt du Gap Agent dit que le classeur est entraine sur des donnees synthetiques ; le RAG repond "I don't know" plutot que d'inventer
Tests	85 tests backend (faux LLM, faux RAG, faux service meteo), 30 tests frontend, tests du moteur d'achat et du Trip Agent ; essais reels de bout en bout avec GPT-4o, le RAG et Open-Meteo

Limites assumees

- Le RAG ne connait pas le placard de l'utilisateur ; il ne contient que des livres de mode.
- Le classeur XGBoost du Gap Agent est entraine sur des donnees synthetiques jusqu'a l'arrivee de retours reels.
- L'AI Stylist n'a pas de memoire entre deux messages ; la memoire du Trip Agent est en memoire serveur (perdue au redemarrage).
- Le Stylist BO4 et l'orchestrateur LangGraph ne sont pas encore relies a l'interface React.
- La meteo est celle d'un jour (jusqu'a 15 jours) pour une seule ville.
- Pistes : orchestrateur client A2A, memoire de preferences par utilisateur, journal de ce qui est porte, migration de l'AI Stylist vers LangGraph.
- A2A : implementation legere du protocole (JSON-RPC, cartes d'agent) ; streaming et notifications push non implementes.

13. Communication entre agents : protocole A2A

Chaque agent expose une **carte d'agent** (qui il est, ses competences, comment s'authentifier) et un point d'entree **JSON-RPC** (protocole Agent2Agent, version 0.3). N'importe quel agent ou l'orchestrateur peut donc le decouvrir et lui confier une tache par HTTP, sans connaitre ses routes internes. Le code commun est dans `a2a_protocol/` (serveur, client).

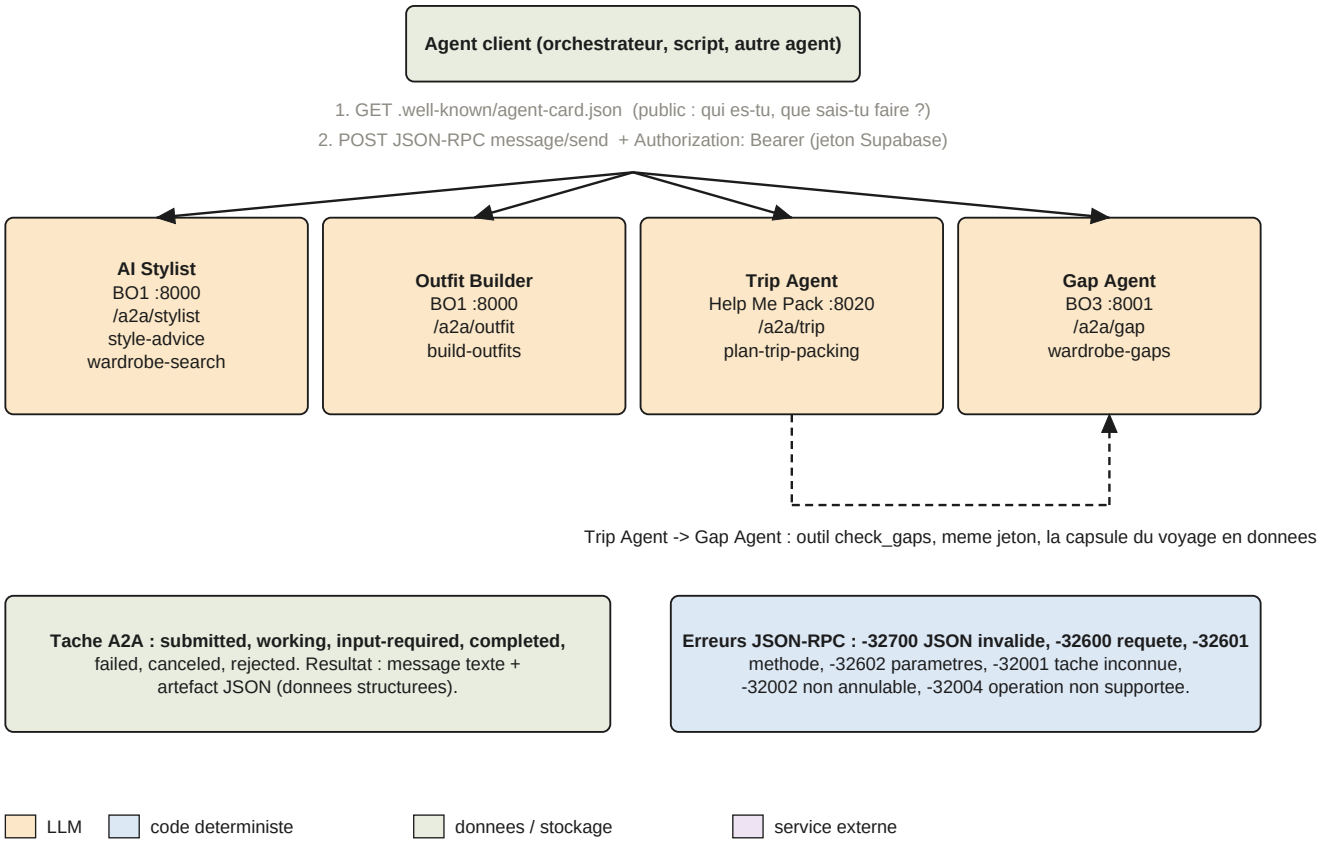


Figure 9 : decouverte par carte, envoi d'une tache, appel d'un agent par un autre.

Agent	Carte d'agent	Point d'entree	Competence (skill)	Donnees acceptees
AI Stylist	/a2a/stylist/.well-known/agent-card.json	POST /a2a/stylist (BO1)	style-advice, wardrobe-search	texte ; city, country, unit
Outfit Builder	/a2a/outfit/.well-known/agent-card.json	POST /a2a/outfit (BO1)	build-outfits	texte ou occasion, season, city, country, avoid
Wardrobe Agent	/a2a/wardrobe/.well-known/agent-card.json	POST /a2a/wardrobe (BO1)	recherche, lecture, correction, stats, similaires, doublons	texte ; action (list, search, check-duplicate), confirm
Trip Agent	/a2a/trip/.well-known/agent-card.json	POST /a2a/trip (8020)	plan-trip-packing	texte ou trip {destination, dates, activities, kg}
Gap Agent	/a2a/gap/.well-known/agent-card.json	POST /a2a/gap (BO3 8001)	wardrobe-gaps	texte ou goal ; capsule du voyage

Ce qui est implemente

- Methodes `message/send` (reponse synchrone : une tache terminee), `tasks/get` et `tasks/cancel`.
- **Authentification** : la carte est publique ; chaque tache exige le jeton Supabase (en-tete Authorization) et ne travaille que sur la garde-robe de cet utilisateur. Une tache n'est lisible que par celui qui l'a creee.
- **Lecture de la garde-robe par A2A** : le Trip Agent demande les pieces au Wardrobe Agent (data.action = list) avec le jeton de l'utilisateur ; si le Wardrobe Agent est indisponible, il retombe sur la route BO1.
- **input-required** : le Trip Agent demande une precision ("quelles dates ?") ; le client renvoie sa reponse avec le meme `contextId` (c'est l'identifiant de la session de voyage).
- Un agent qui plante renvoie une tache **failed** avec un message neutre : aucun detail interne ne sort.
- Non implemente, et declare tel quel dans la carte : streaming (`message/stream`) et notifications push.

Agent vers agent : le Trip Agent demande au Gap Agent

L'outil `check_gaps` du Trip Agent etait un crochet vide. Il appelle maintenant le Gap Agent **par A2A** : il lui envoie la capsule du voyage (destination, activites, jours) et le jeton de l'utilisateur ; le Gap Agent choisit l'objectif le plus proche, classe les pieces manquantes et repond ; le Trip Agent les propose. Si le Gap Agent est arrete, l'outil renvoie une erreur propre et le voyage n'est pas bloque. L'adresse se regle avec `GAP_A2A_URL` (vide pour desactiver).

Decouvrir les agents

`python scripts/a2a_discover.py` lit les cinq cartes ; avec `A2A_TOKEN` et `--ask`, il envoie aussi une tache a chaque agent.

Suite logique

L'orchestrateur devient un **client A2A** : il lit les cinq cartes, choisit l'agent selon les competences annoncees et lui delegue la tache. Les agents sont deja prets pour cela.